

Weekly Progress: Validation, Resampling, and Decision-Level Explanation

June 3, 2026

1 Overview

This week’s work follows the three-step plan discussed after the previous meeting. First, I used PyEPO as an external reference implementation to check whether my current codebase contains implementation bugs. Second, I ran subset-resampling experiments to summarize variability with box plots. Third, assuming the implementation is reliable, I started a decision-level analysis to understand why the two-stage model can sometimes remain close to SPO+/FY in decision quality.

The resulting workflow is:

Step	Goal	Current status
1	Implementation validation	Passed for LR; SPO+ algebra/gradient/trajectory validated
2	Subset-resampling variability	Box plots generated on fixed graph pools
3	Decision-level explanation	First-pass analysis completed on Step2b d8

2 Step 1: Implementation Validation with PyEPO Reference

2.1 Motivation

The goal of this step is to check whether my implementation contains subtle bugs. The logic is simple: if my code cannot reproduce PyEPO behavior on a standard benchmark or on the same KEP oracle, then there may be a bug in the loss, gradient, optimizer update, solver interface, or metric calculation.

2.2 Shortest-Path Benchmark

I first used PyEPO’s fixed-data shortest-path setup as an external benchmark. For the two-stage LR baseline, my implementation fully matches PyEPO across all fixed-data shortest-path settings:

For SPO+, the loss, gradient, and one-step update match PyEPO when the oracle solution is shared. The remaining full-CSV differences are explained by shortest-path tie-breaking: the same cost vector can have multiple optimal paths, and PyEPO’s `optDataset` can store a different optimal solution vector across repeated solves even when the optimal objective is identical. Since SPO+ gradients depend on the solution vector and not only on the objective value, this can create training divergence without implying an algebraic bug.

Table 1: Shortest-path PyEPO-vs-my comparison.

Quantity	PyEPO LR vs my LR	PyEPO SPO+ vs my SPO+
Result files	24 / 24	24 / 24
Rows	240 / 240	240 / 240
Passing settings	24 / 24	19 / 24
True SPO global max diff	3.12×10^{-19}	3.61×10^{-4}
Unambiguous SPO global max diff	2.07×10^{-19}	3.61×10^{-4}
MSE global max diff	1.09×10^{-47}	2.70×10^{-3}
Epoch diff	0	0

2.3 KEP Reference Validation

I then moved the validation to the actual KEP oracle. This is more directly relevant to Step1c because KEP is a reward-maximization problem with instance-specific graph structure. The KEP validation uses the same `CachedHybridKepModel` / Gurobi oracle and compares PyEPO-style SPOPlus reference computations with my Step1c reward-max SPO+ implementation.

Table 2: KEP PyEPO-reference vs Step1c validation summary.

Validation phase	Key comparison	Result
Phase 0 preflight	reward-max / cost-min sign adapter	all max diffs = 0
Phase 1 small setting	SPO+ forward loss	max diff = 2.9998×10^{-6}
Phase 1 small setting	SPO+ gradient and one-step SGD update	max diff = 0
Phase 2 full trajectory	$\theta_1, \theta_2, \ \theta\ $	max diff = 0
Phase 2 full trajectory	train SPO+ loss	max diff = 3.2975×10^{-6}
Phase 2 full trajectory	validation SPO+ loss	max diff = 2.2324×10^{-6}
Phase 2 full trajectory	train / validation decision gap	max diff = 0

2.4 Conclusion from Step 1

The implementation check supports the conclusion that the Step1c SPO+ algebra, reward-max sign convention, gradient scaling, optimizer update, KEP oracle interface, and evaluation path are consistent with an independent PyEPO-based reference implementation. The remaining shortest-path SPO+ CSV differences are attributable to oracle tie-breaking among multiple shortest paths, not to an identified error in the Step1c SPO+ formula.

3 Step 2: Subset-Resampling Variability

3.1 Motivation

The second step is to turn a single-seed observation into distributional evidence. The current resampling experiment intentionally isolates one source of randomness: the training subset. The graph pool, label regime, validation set, heldout set, and training initialization are fixed. Only the subset seed used to sample 50 training graphs is varied.

Thus, this experiment should be interpreted as:

Train-subset resampling on a fixed graph pool and fixed label regime.

It does not yet test graph-generation seed, label-noise seed, or training initialization seed.

3.2 Experimental Setup

The first-pass resampling setting is:

Item	Setting
Train size	50
Randomness varied	subset seed only
Graph pool	fixed
Training seed	fixed
Evaluation	heldout400
Main methods	2stage selected by validation MSE; SPO+ selected by validation SPO+ loss
Primary metric	mean normalized decision gap

3.3 Generated Plots

The current plotting script summarizes the heldout400 distribution with four main outputs:

- overall gap boxplot;
- by-block gap boxplot;
- paired reduction boxplot;
- relative reduction by block.

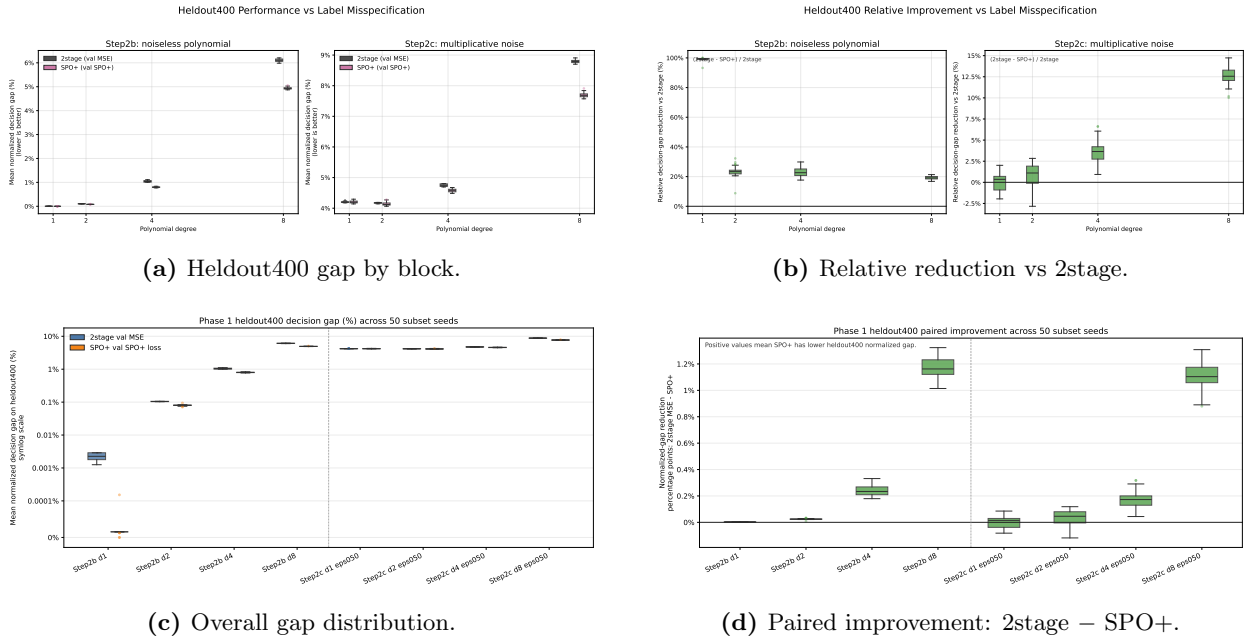


Figure 1: Subset-resampling box plots on heldout400. Positive paired reduction means SPO+ has a lower normalized decision gap than 2stage on the same subset seed.

3.4 Interpretation

These plots answer whether the SPO+ improvement is stable under different sampled training subsets from the same fixed graph pool. The paired reduction plot is especially important because it compares 2stage and SPO+ on the same subset seed, reducing confounding from subset difficulty.

The correct conclusion is limited but useful:

SPO+ improvement appears stable under train-subset resampling on the fixed Step2 graph pool, especially in high-degree misspecification regimes.

The current experiment should not be described as full robustness over all randomness. A stronger next-stage robustness test would vary graph-generation seed, label-noise seed, or training initialization seed.

4 Step 3: Decision-Level Explanation

4.1 Motivation

After validating the implementation and observing subset-resampling variability, I investigated why 2stage can remain competitive with SPO+. The hypothesis is that not all edge-level prediction errors are decision-critical. Some large errors occur on edges that never enter the selected KEP solution, while other selected solutions may be near-optimal alternatives.

4.2 Scope

The first-pass decision analysis focuses on:

Step2b d8, $n_{\text{train}} = 50$, heldout400.

It compares:

2stage selected by validation MSE vs. SPO+ selected by validation SPO+ loss.

Nine representative subset seeds were selected:

$\{1, 25, 22\}$ large improvement, $\{27, 21, 30\}$ weak/borderline, $\{41, 16, 33\}$ easy low-gap.

4.3 Data Products

The analysis produced:

Output	Rows / files
selected case seeds	9 rows
per-graph decision comparison	7,200 rows
edge-level criticality table	879,642 rows
graph-level criticality summary	7,200 rows
case-study index	9 rows
case-study edge tables	9 tables
margin / near-tie analysis	3,600 rows
observed candidate ranking	10,800 rows
decision-critical MSE correlations	12 rows

Table 3: Decision-level summary on selected Step2b d8 heldout400 cases.

Metric	2stage	SPO+
Mean normalized decision gap	0.06087	0.04938
Mean Jaccard with oracle solution	0.60580	0.63655
Exact oracle-solution rate	17.81%	23.58%
Top-10-error edges selected rate	22.87%	19.05%
Top-10-error edges in symmetric difference	9.39%	7.02%
<i>Observed 2stage/SPO+ solution relationship</i>		
Same 2stage/SPO+ solution rate	55.08%	
Mean Jaccard between 2stage and SPO+	0.82707	
Median Jaccard between 2stage and SPO+	1.00000	
Any-method near-tie rate, threshold = 0.01	9.17%	
<i>Correlation with normalized gap, Pearson / Spearman</i>		
All-edge MSE	0.095 / 0.023	-0.021 / -0.075
MSE on symmetric-difference edges	0.390 / 0.528	0.272 / 0.372
Top-10-error symmetric-difference rate	0.570 / 0.667	0.557 / 0.639

4.4 Summary Statistics

4.5 Main Finding

Raw edge-level MSE is weakly correlated with decision gap. In contrast, symmetric-difference-focused measures are much more predictive. This suggests that prediction error matters most when it lands on decision-changing edges, not merely because an edge-level error is large.

4.6 Case Studies

Case A: Bad prediction but irrelevant.

Seed	Graph	2stage gap	All-edge MSE	Top-10-error symdiff rate
41	G-234.json	0	19.6328	0.0
41	G-994.json	0	18.1776	0.0
25	G-1516.json	0	17.7713	0.0

Large prediction errors can occur on edges that do not affect the selected KEP solution. These examples have high all-edge MSE but zero decision gap and zero top-error symmetric-difference involvement.

Figure 2: Large edge-level prediction errors can be non-critical.

4.7 Conclusion from Step 3

The first-pass decision analysis supports the advisor’s hypothesis. The competitiveness of 2stage does not mean that it predicts every edge accurately. Instead, many large edge-level errors are non-critical because they do not change the selected exchange. In addition, some different KEP solutions are near-optimal alternatives under the true objective. SPO+ appears most useful when prediction errors affect edges in the symmetric difference between the predicted and oracle solutions.

Case B: Different solution but near-optimal.

Seed	Graph	2stage gap	Jaccard with oracle	Same as oracle?
1	G-696.json	0.000644	0.6818	No
1	G-1372.json	0.001174	0.6087	No
1	G-607.json	0.001175	0.7143	No

The selected edge set can differ from the oracle solution while the objective loss remains nearly zero. This indicates that some KEP graphs contain near-optimal alternative exchanges.

Figure 3: Different selected solutions can be near-optimal alternatives.

Case C: SPO+ fixes 2stage decision-critical errors.

Seed	Graph	2stage gap	SPO+ gap	2stage top-10 symdiff	SPO+ top-10 symdiff
1	G-392.json	0.313849	0	0.3	0.0
30	G-1560.json	0.292305	0.011964	0.3	0.0
1	G-39.json	0.316226	0.080459	0.2	0.1

SPO+ helps when prediction errors affect decision-changing edges. In these examples, SPO+ substantially reduces decision gap and also reduces high-error symmetric-difference involvement.

Figure 4: SPO+ improves decisions when errors are decision-critical.

5 Overall Conclusions and Next Steps

5.1 Conclusions

This week’s work forms a closed loop:

1. **Code validation.** The Step1c SPO+ implementation aligns with PyEPO reference computations on both shortest-path and KEP oracles. The remaining shortest-path full-training SPO+ discrepancy is explained by oracle tie-breaking among multiple shortest paths.
2. **Distributional evidence.** The subset-resampling box plots show how 2stage and SPO+ vary across different 50-graph training subsets on a fixed graph pool.
3. **Mechanistic explanation.** The decision-level analysis suggests why 2stage can remain competitive: many prediction errors are not decision-critical, and some different selected exchanges are near-optimal. SPO+ improves most when errors occur on decision-changing edges.

5.2 Limitations

The resampling analysis currently varies only the training subset seed. It does not yet vary graph-generation seed, label-noise seed, or training initialization seed. The decision-level analysis is currently limited to **Step2b d8**, train size 50, heldout400, selected representative seeds, and 2stage vs SPO+. It does not yet include FY, **Step2c d8 eps050**, full graph-seed variation, or adversarial SPO+ solutions y_{adv} .

5.3 Next Steps

The next report should extend the explanation in three directions:

1. Apply the same decision-level pipeline to **Step2c d8 eps050** to check whether the same non-critical-error mechanism holds under multiplicative label noise.
2. Add FY as a third method and compare whether FY-selected solutions are closer to SPO+, 2stage, or the oracle solution.
3. Add y_{adv} analysis to inspect which edges the SPO+ gradient pushes away from or toward.